

Module 4: Frequent Pattern & Association Mining

Topics 30 to 36 • Market Basket Analysis, Support/Confidence/Lift, Apriori, FP-Growth & ECLAT

Module 4 Table of Contents (Topics 30 to 36)

- **Topic 30:** Market Basket Analysis & Association Rules
- **Topic 31:** Support, Confidence, Lift, Conviction Metrics
- **Topic 32:** Apriori Algorithm (Join & Prune Steps)
- **Topic 33:** FP-Growth Algorithm (FP-Tree Construction)
- **Topic 34:** ECLAT Vertical Data Format Algorithm
- **Topic 35:** Closed vs. Maximal Frequent Itemsets
- **Topic 36:** 15 Solved University Examination Bank

Topic 30 & 31: Market Basket Analysis & Association Rule Metrics

Frequent Pattern Mining searches for recurring itemsets, subsequences, or substructures in transactional databases. In **Market Basket Analysis**, retailers identify customer purchasing habits by discovering associations between items placed simultaneously in shopping carts.

The 5 Fundamental Association Rule Quality Metrics ($A \Rightarrow B$):

$$\text{Support}(A \Rightarrow B) = P(A \cup B) = \frac{\text{count}(A \cup B)}{|D|}$$

$$\text{Confidence}(A \Rightarrow B) = P(B | A) = \frac{P(A \cup B)}{P(A)} = \frac{\text{count}(A \cup B)}{\text{count}(A)}$$

$$\text{Lift}(A \Rightarrow B) = \frac{P(A \cup B)}{P(A)P(B)} = \frac{\text{Confidence}(A \Rightarrow B)}{\text{Support}(B)}$$

$$\text{Conviction}(A \Rightarrow B) = \frac{P(A)P(\neg B)}{P(A \cup \neg B)} = \frac{1 - \text{Support}(B)}{1 - \text{Confidence}(A \Rightarrow B)}$$

$$\text{Leverage}(A \Rightarrow B) = P(A \cup B) - P(A)P(B) = \text{Support}(A \Rightarrow B) - \text{Support}(A)\text{Support}(B)$$

Interpretation of Lift: - Lift = 1: A and B are statistically independent (rule has zero predictive value). - Lift > 1: A and B are **positively correlated** (buying A boosts the likelihood of buying B). - Lift < 1: A and B are **negatively correlated** (substitutes; buying A decreases buying B).

Step-by-Step Solved Problem: Association Rule Metric Calculation

A supermarket records $N = 10,000$ transactions. Bread is bought in 6,000 transactions, Butter in 7,500 transactions, and both Bread and Butter together in 4,500 transactions. Evaluate the rule $\text{Bread} \implies \text{Butter}$:

$$\text{Support}(\text{Bread} \implies \text{Butter}) = \frac{4500}{10000} = \mathbf{0.45 = 45\%}$$

$$\text{Confidence}(\text{Bread} \implies \text{Butter}) = \frac{\text{count}(\text{Bread} \cup \text{Butter})}{\text{count}(\text{Bread})} = \frac{4500}{6000} = \mathbf{0.75 = 75\%}$$

$$\text{Lift}(\text{Bread} \implies \text{Butter}) = \frac{\text{Confidence}}{\text{Support}(\text{Butter})} = \frac{0.75}{7500/10000} = \frac{0.75}{0.75} = \mathbf{1.00}$$

Critical Insight: Even though Confidence is high (75%), Lift = 1.00 proves Bread and Butter are INDEPENDENT!

Customers buy butter 75% of the time anyway, regardless of whether they bought bread! The rule is deceptive!

Topic 32: The Apriori Algorithm (Join & Prune Steps)

The Apriori Principle & Anti-Monotonicity of Support

Core Axiom: "If an itemset is frequent, then all of its subsets must also be frequent."

$$\forall \mathbf{X}, \mathbf{Y} : (\mathbf{X} \subseteq \mathbf{Y}) \implies \text{Support}(\mathbf{X}) \geq \text{Support}(\mathbf{Y})$$

Contrapositive (Pruning Rule): "If an itemset S is infrequent ($\text{Support}(S) < \text{min_sup}$), then none of its supersets can ever be frequent."

🏠 Step-by-Step Solved Problem: Complete Apriori Execution Trace

Transactional Database ($N = 5$), $\min_sup = 2$ (40%):

- $T_1 : \{A, C, D\}$
- $T_2 : \{B, C, E\}$
- $T_3 : \{A, B, C, E\}$
- $T_4 : \{B, E\}$
- $T_5 : \{A, B, C, E\}$

Pass 1: Candidate 1-itemsets $C_1 : \{A : 3, B : 4, C : 4, D : 1, E : 4\}$.

Prune D ($1 < 2$) $\implies L_1 = \{\{A\} : 3, \{B\} : 4, \{C\} : 4, \{E\} : 4\}$.

Pass 2: Candidate 2-itemsets $C_2 = L_1 \bowtie L_1 : \{AB, AC, AE, BC, BE, CE\}$.

Scan DB: $\{AB : 2, AC : 3, AE : 2, BC : 3, BE : 4, CE : 3\}$. All $\geq 2 \implies L_2 = C_2$.

Pass 3: Candidate 3-itemsets $C_3 = L_2 \bowtie L_2$:

- Join $\{AB\} \bowtie \{AC\} \rightarrow \{ABC\}$. Subsets: $AB, AC, BC \in L_2$ (Valid!).

- Join $\{AB\} \bowtie \{AE\} \rightarrow \{ABE\}$. Subsets: $AB, AE, BE \in L_2$ (Valid!).

- Join $\{AC\} \bowtie \{AE\} \rightarrow \{ACE\}$. Subsets: $AC, AE, CE \in L_2$ (Valid!).

- Join $\{BC\} \bowtie \{BE\} \rightarrow \{BCE\}$. Subsets: $BC, BE, CE \in L_2$ (Valid!).

Scan DB for $C_3 : \{ABC : 2, ABE : 2, ACE : 2, BCE : 3\}$. All $\geq 2 \implies L_3 = C_3$.

Pass 4: Candidate 4-itemset $C_4 = L_3 \bowtie L_3 : \{ABCE\}$.

Prune Check: Subset $\{BCD\} \dots$ Subsets: $\{ABC\}, \{ABE\}, \{ACE\}, \{BCE\}$ all in L_3 . Valid!

Scan DB for $\{ABCE\}$: Appears in $T_3, T_5 \implies \text{count} = 2 \geq 2 \implies L_4 = \{\{A, B, C, E\} : 2\}$.

Final Max Frequent Itemset: $\{A, B, C, E\}$ with Support = 40%

Topic 33 & 34: FP-Growth Algorithm & ECLAT Vertical Format

Algorithm	Data Representation	Mining Mechanism	Key Advantages
Apriori (Agrawal 1994)	Horizontal transaction format $\langle \text{TID}, \text{Itemset} \rangle$.	Level-wise BFS candidate generation ($L_k \bowtie L_k$) and repeated database disk scans (k scans).	Simple; high memory overhead for candidate itemsets ($O(2^{ I })$).
FP-Growth (Han et al. 2000)	Compact tree structure (FP-Tree) with header table node links.	Divide-and-conquer recursive mining of conditional pattern bases; Zero candidate generation!	Only 2 database scans required; orders of magnitude faster than Apriori!
ECLAT (Zaki 2000)	Vertical data format $\langle \text{Item}, \text{TID-List} \rangle$.	Intersection of TID lists ($\text{TID}(AB) = \text{TID}(A) \cap \text{TID}(B)$).	No candidate generation; fast bitwise AND operations; large memory for long TID lists.

Topic 35 & 36: Closed vs. Maximal Itemsets & Solved Exam Questions

Closed vs. Maximal Frequent Itemsets: - An itemset X is **Closed** if X is frequent and there exists NO proper superset $Y \supset X$ such that $\text{Support}(Y) = \text{Support}(X)$. (Preserves complete support information without loss!). - An itemset X is **Maximal** if X is frequent and there exists NO proper superset $Y \supset X$ that is frequent ($\text{Support}(Y) \geq \text{min_sup}$).
(Lossy compression; $|Maximal| \leq |Closed| \leq |Frequent|$).

Q1. Trace the FP-Tree Construction Algorithm for transactions: $T_1 = \{a, b, c\}, T_2 = \{b, c, d\}, T_3 = \{a, c, d, e\}$ with $\text{min_sup} = 2$. (10 Marks)

1. Scan DB to find frequencies: $c : 3, a : 2, b : 2, d : 2, e : 1$.
2. Filter and sort by descending frequency: $L_1 = [c, a, b, d]$ (Item e pruned).
3. Re-order transactions: $T_1 \rightarrow [c, a, b], T_2 \rightarrow [c, b, d], T_3 \rightarrow [c, a, d]$.
4. Insert into FP-Tree from root 'null':
 - Insert T_1 : 'null' $\rightarrow (c : 1) \rightarrow (a : 1) \rightarrow (b : 1)$.
 - Insert T_2 : 'null' $\rightarrow (c : 2) \rightarrow (b : 1) \rightarrow (d : 1)$.
 - Insert T_3 : 'null' $\rightarrow (c : 3) \rightarrow (a : 2) \rightarrow (d : 1)$.
5. Construct Header Table with linked lists connecting identical item nodes across branches!

Topic 36.1: Advanced Sequence Pattern Mining & Negative Association Rules

Negative Association Rules & Correlation: A negative association rule takes the form $A \Rightarrow \neg B$ or $\neg A \Rightarrow B$. It is considered statistically valid and interesting if:

$$\text{Support}(A \cup \neg B) = \text{Support}(A) - \text{Support}(A \cup B) \geq \text{min_sup}$$

$$\text{Confidence}(A \Rightarrow \neg B) = \frac{\text{Support}(A \cup \neg B)}{\text{Support}(A)} = 1 - \text{Confidence}(A \Rightarrow B) \geq \text{min_conf}$$

$$\text{Correlation Measure: } \text{Corr}(A, B) = \frac{P(A \cup B)}{P(A)P(B)} < 1 \quad (\text{Substitutability Indicator})$$

Step-by-Step Solved Problem: Generalized Sequential Pattern (GSP) Trace

Given sequence database: $S_1 = \langle (a)(b, c)(d) \rangle, S_2 = \langle (a, d)(c)(b) \rangle, S_3 = \langle (a)(c)(d) \rangle$. Minimum support = 2 (66.7%):

- Frequent 1-sequences L_1 : $\langle (a) \rangle : 3, \langle (b) \rangle : 2, \langle (c) \rangle : 3, \langle (d) \rangle : 3$.
- Candidate 2-sequences C_2 : $\langle (a)(c) \rangle, \langle (a, c) \rangle, \langle (c)(d) \rangle, \langle (a)(d) \rangle \dots$
- Scan DB: $\langle (a)(c) \rangle$ appears in $S_1, S_3 \Rightarrow \text{count} = 2$; $\langle (a)(d) \rangle$ appears in $S_1, S_3 \Rightarrow \text{count} = 2$.
- Frequent 3-sequence $L_3 = \{ \langle (a)(c)(d) \rangle : 2 \}$.

Q2. Compare the PrefixSpan (Prefix-Projected Pattern Growth) Algorithm with GSP. (8 Marks)

- **GSP (Srikant & Agrawal):** An Apriori-like breadth-first sequence mining algorithm. Generates massive sets of candidate sequences ($L_{k-1} \bowtie L_{k-1}$) and scans the database repeatedly, incurring severe disk I/O bottlenecks.
- **PrefixSpan (Pei et al.):** A pattern-growth algorithm that mines sequential patterns by recursively projecting sequence suffixes into smaller projected databases based on frequent prefixes. **Zero candidate generation**; dramatically smaller memory footprint and orders of magnitude faster execution!

Topic 36.2: Advanced Constraint-Based Mining & Multi-Level Association Rules

Constraint Property	Formal Mathematical Invariant	Optimization in Mining Algorithms
Anti-Monotone	$\forall S \supset S' : \text{If } S' \text{ violates constraint } C \implies S \text{ must also violate } C.$	Can be pushed deep into candidate generation (e.g. $\text{sum}(S.\text{price}) \leq 500$, $\text{Support}(S) \geq \text{min_sup}$).
Monotone	$\forall S \supset S' : \text{If } S' \text{ satisfies constraint } C \implies S \text{ must also satisfy } C.$	Once an itemset satisfies constraint (e.g. $\text{sum}(S.\text{price}) \geq 1000$), all its supersets are guaranteed valid!
Succinct	All and only itemsets satisfying C can be explicitly generated upfront without testing (e.g. $\text{Item.Type} = \text{'Electronics'}$).	Pre-filters data before any mining begins; zero candidate testing overhead!
Convertible	A non-monotonic constraint becomes monotone or anti-monotone when items are arranged in a specific sorted order (e.g. $\text{avg}(S.\text{price}) \geq 50$ sorted by descending price).	Enables itemset pruning when combined with FP-Growth header sorting.

Step-by-Step Solved Problem: Multi-Level Association Rule Redundancy Filtering

Consider a product hierarchy where `Laptop` is a parent category of `Dell\_XPS` and `Apple\_MacBook`:

- Rule 1 (High-Level): $\text{Laptop} \implies \text{Mouse}$ [Support = 10%, Confidence = 70%]
- Rule 2 (Low-Level): $\text{Dell_XPS} \implies \text{Mouse}$ [Support = 3%, Confidence = 72%]
- Rule 3 (Low-Level): $\text{Apple_MacBook} \implies \text{Mouse}$ [Support = 4%, Confidence = 20%]

Redundancy Analysis:

- Rule 2 has confidence (72%) almost identical to its generalized ancestor Rule 1 (70%). It provides NO new unexpected information $\implies$ **Rule 2 is REDUNDANT and should be pruned!**
- Rule 3 has confidence (20%) drastically lower than ancestor Rule 1 (70%). It reveals a vital unexpected negative insight (Mac users buy trackpads or don't buy mice) $\implies$ **Rule 3 is HIGHLY INTERESTING and must be preserved!**

Topic 36.3: Master University Exam Problem Bank (Part II)

Solved Problem 4: Quantitative Association Rule Mining & Equi-Depth Binning

When transactional databases contain continuous numeric attributes (e.g., $\text{Age} \in [18, 80]$, $\text{Income} \in [\$10\text{k}, \$200\text{k}]$):

- Continuous attributes are discretized into intervals: $\text{Age} \in [20, 30], [31, 40], \dots$
- **Equi-Width Binning:** Divides the range into k equal intervals ($\text{Width} = \frac{\text{max} - \text{min}}{k}$). Prone to severe skew if data clusters in few bins.
- **Equi-Depth (Equal-Frequency) Binning:** Divides data such that each bin contains exactly N/k tuples, guaranteeing equal statistical representation!
- **Rule Extraction:** $\text{Age} \in [20, 29] \wedge \text{Income} \in [\$40\text{k}, \$60\text{k}] \implies \text{BuysCar: 'Yes'}$.

Q3. Explain Rare Itemset Mining and why standard Apriori fails to discover rare patterns. (8 Marks)

In domains like Credit Card Fraud Detection or Rare Disease Diagnosis, critical patterns occur with extremely low frequency (e.g., 0.01% support).

- **The Rare Item Problem:** If `min_support` is set high, rare fraudulent patterns are pruned and lost. If `min_support` is set very low, Apriori suffers combinatorial explosion generating millions of uninteresting common itemset candidates!
- **Solution (MSApriori):** Multiple Minimum Support Apriori assigns individual Minimum Item Support (MIS) thresholds to each item ($MIS(i)$), allowing rare items to participate in candidate generation with low thresholds while common items require high thresholds!

Q4. Explain Hash-Based Itemset Counting in the DHP (Direct Hashing and Pruning) Algorithm. (8 Marks)

DHP (Park et al. 1995) accelerates Apriori Pass 2 by using a hash table to filter out candidate 2-itemsets during the Pass 1 database scan:

1. When reading a transaction T during Pass 1, generate all 2-itemset combinations in T and hash them into a fixed hash bucket table ($h(x, y) = (x \cdot 10 + y) \pmod{H}$).
 2. Increment the bucket count.
 3. In Pass 2, when joining $L_1 \bowtie L_1$, only generate candidate pair $\{A, B\}$ if its corresponding hash bucket count $\geq \text{min_sup}$!
- This prunes up to 90% of invalid 2-itemset candidates before the Pass 2 database scan even starts!

Topic 36.4: Master University Exam Problem Bank (Part III)

Solved Problem 5: Complete FP-Tree Conditional Pattern Base Mining Trace

Consider frequent items in descending order: $L_1 = [f : 4, c : 4, a : 3, b : 3, m : 3, p : 3]$. Suppose we are mining patterns ending in suffix p :

- Paths leading to p in FP-Tree: $\langle f, c, a, m : 2 \rangle$ and $\langle c, b : 1 \rangle$.
- **Conditional Pattern Base for p :** $\{(f, c, a, m) : 2, (c, b) : 1\}$.
- Accumulate item frequencies: $f : 2, c : 3, a : 2, m : 2, b : 1$.
- With `min_sup` = 3, items f, a, m, b are pruned (< 3). Only item c remains (`count` = $3 \geq 3$).
- **Conditional FP-Tree for p :** $\langle c : 3 \rangle$.
- **Generated Frequent Patterns ending in p :** $\{p\} : 3$ and $\{c, p\} : 3$.

Result: Zero candidate generation required! Directly mined frequent itemset $\{c, p\}$ in $O(1)$ time!

Q5. Explain the Apriori-Tid and Apriori-Hybrid Algorithms. (8 Marks)

- **Apriori-Tid:** Does NOT scan the raw transactional database after Pass 1. Instead, it generates a synthetic transactional table $\bar{C}_k$ storing tuple IDs alongside candidate itemset IDs. In late passes ($k \geq 3$), $\bar{C}_k$ becomes dramatically smaller than the original database, accelerating support counting.
- **Apriori-Hybrid:** Uses standard Apriori in early passes ($k = 1, 2$) when $\bar{C}_k$ is larger than the raw database, and automatically switches to Apriori-Tid in later passes ($k \geq 3$) when candidate itemset lists shrink, achieving optimal execution speed!

 Solved Problem 6: Vertical Data Format Mining with ECLAT Algorithm Trace

Given transactional database ($N = 4$): $T_1 = \{A, B, C\}, T_2 = \{B, C, D\}, T_3 = \{A, C, D\}, T_4 = \{A, B, C, D\}$ with $\min_sup = 2$ (50%):

- **Vertical TID-Lists ($k = 1$):**
 - $t(A) = \{1, 3, 4\}$ (count = 3)
 - $t(B) = \{1, 2, 4\}$ (count = 3)
 - $t(C) = \{1, 2, 3, 4\}$ (count = 4)
 - $t(D) = \{2, 3, 4\}$ (count = 3)
- **2-Itemset TID-List Intersections ($k = 2$):**
 - $t(AB) = t(A) \cap t(B) = \{1, 3, 4\} \cap \{1, 2, 4\} = \{1, 4\} \implies \text{count} = 2 \geq 2$ (Frequent!)
 - $t(AC) = t(A) \cap t(C) = \{1, 3, 4\} \cap \{1, 2, 3, 4\} = \{1, 3, 4\} \implies \text{count} = 3 \geq 2$ (Frequent!)
 - $t(AD) = t(A) \cap t(D) = \{1, 3, 4\} \cap \{2, 3, 4\} = \{3, 4\} \implies \text{count} = 2 \geq 2$ (Frequent!)
 - $t(BC) = t(B) \cap t(C) = \{1, 2, 4\} \cap \{1, 2, 3, 4\} = \{1, 2, 4\} \implies \text{count} = 3 \geq 2$ (Frequent!)
 - $t(BD) = t(B) \cap t(D) = \{1, 2, 4\} \cap \{2, 3, 4\} = \{2, 4\} \implies \text{count} = 2 \geq 2$ (Frequent!)
 - $t(CD) = t(C) \cap t(D) = \{1, 2, 3, 4\} \cap \{2, 3, 4\} = \{2, 3, 4\} \implies \text{count} = 3 \geq 2$ (Frequent!)
- **3-Itemset Intersections ($k = 3$):**
 - $t(ABC) = t(AB) \cap t(AC) = \{1, 4\} \cap \{1, 3, 4\} = \{1, 4\} \implies \text{count} = 2 \geq 2$
 - $t(ABD) = t(AB) \cap t(AD) = \{1, 4\} \cap \{3, 4\} = \{4\} \implies \text{count} = 1 < 2$ (Pruned!)
 - $t(ACD) = t(AC) \cap t(AD) = \{1, 3, 4\} \cap \{3, 4\} = \{3, 4\} \implies \text{count} = 2 \geq 2$
 - $t(BCD) = t(BC) \cap t(BD) = \{1, 2, 4\} \cap \{2, 4\} = \{2, 4\} \implies \text{count} = 2 \geq 2$

Frequent 3-Itemsets: $\{A, B, C\}, \{A, C, D\}, \{B, C, D\}$ (Computed in zero database scans!)

Q6. Explain the Concept of Correlation Rules and Chi-Square Association Testing. (8 Marks)

Because Support-Confidence association rules can produce misleading deceptive correlations (when the consequent item is globally popular), **Correlation Rules** augment association mining by testing statistical independence using the Chi-Square metric:

$$\chi^2 = \sum_{i \in \{A, \neg A\}} \sum_{j \in \{B, \neg B\}} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} = \frac{N(O_{11}O_{22} - O_{12}O_{21})^2}{(O_{11} + O_{12})(O_{21} + O_{22})(O_{11} + O_{21})(O_{12} + O_{22})}$$

If $\chi^2 > 3.841$ (at $\alpha = 0.05, df = 1$), the association rule $A \implies B$ is proven to possess genuine statistical correlation rather than accidental co-occurrence!

Topic 36.6: Complete Solved Laboratory Problem Bank on Association Rules

Solved Numerical 7: Association Rule Conviction & Leverage Calculations

In a database of 1000 transactions, $\text{Support}(A) = 0.20$, $\text{Support}(B) = 0.60$, $\text{Support}(A \cup B) = 0.18$:

- $\text{Confidence}(A \Rightarrow B) = \frac{0.18}{0.20} = \mathbf{0.90} = \mathbf{90\%}$
- $\text{Lift}(A \Rightarrow B) = \frac{0.90}{0.60} = \mathbf{1.50} > 1 \Rightarrow \text{Positive correlation}$
- **Leverage:** $\text{Leverage} = P(A \cup B) - P(A)P(B) = 0.18 - (0.20)(0.60) = 0.18 - 0.12 = +\mathbf{0.06}$
- **Conviction:** $\text{Conviction} = \frac{1 - P(B)}{1 - \text{Conf}(A \Rightarrow B)} = \frac{1 - 0.60}{1 - 0.90} = \frac{0.40}{0.10} = \mathbf{4.00}$
- $\text{Conviction} = 4.00$ implies that the rule would be incorrect 4 times more often if the association were purely accidental!

Solved Problem 11: Multi-Dimensional Association Rule Mining on Hybrid Relational Data

When mining rules across multiple database relations (e.g. 'Customer', 'Account', 'Transaction'):

Rule: $\text{age}(X, "20..29") \wedge \text{income}(X, "40K..60K") \Rightarrow \text{buys}(X, "HDTV")$ [Support=2%, Confidence=60%]

- **Quantitative Attributes Discretization:** Age and Income are mapped to discrete intervals using equi-depth binning.
- **Predicate Itemset Representation:** Each (attribute, interval) pair is treated as an item in a multidimensional transaction table.

Solved Problem 12: Sequence Pattern Mining with PrefixSpan Projected Databases

PrefixSpan mines sequential patterns by projecting sequence suffixes into smaller projected databases, eliminating candidate generation entirely!

Solved Numerical 11: Quantitative Association Mining with Clustering Pre-Discretization

Instead of arbitrary binning, cluster numeric values using 1D k -Means, replacing continuous coordinates with cluster membership labels before applying Apriori!

Q7. Explain Maximal Frequent Itemsets vs Closed Frequent Itemsets with Concrete Examples. (8 Marks)

Given frequent itemsets with supports: $\{A\} : 4$, $\{B\} : 4$, $\{C\} : 3$, $\{A, B\} : 4$, $\{A, C\} : 2$ with $\text{min_sup} = 2$:

- **Closed Itemset:** $\{A, B\}$ is closed (support 4) because no superset has support 4. $\{A\}$ is NOT closed because superset $\{A, B\}$ has the same support (4). $\{A, C\}$ is closed (support 2).
- **Maximal Itemset:** $\{A, B\}$ and $\{A, C\}$ are maximal because no supersets are frequent. Maximal representation is compact, but loses intermediate subset support counts!

Solved Problem 13: Correlation Rule Mining via Lift and Chi-Square Testing

In a dataset of $N = 1000$ retail transactions, $\text{count}(A) = 400$, $\text{count}(B) = 700$, $\text{count}(A \cup B) = 350$:

- $\text{Support}(A \implies B) = \frac{350}{1000} = \mathbf{35\%}$
- $\text{Confidence}(A \implies B) = \frac{350}{400} = \mathbf{87.5\%}$
- $\text{Lift}(A \implies B) = \frac{0.875}{0.700} = \mathbf{1.25} > 1 \implies$ Genuine positive correlation!
- $\text{Leverage} = 0.35 - (0.40)(0.70) = 0.35 - 0.28 = \mathbf{+0.07}$
- $\text{Conviction} = \frac{1-0.70}{1-0.875} = \frac{0.30}{0.125} = \mathbf{2.40}$

Q8. Detail the ECLAT Algorithm vs Apriori in Terms of Memory and Disk I/O. (8 Marks)

- **Apriori**: Low memory footprint per pass, but incurs heavy repeated disk I/O scanning the massive transactional database k times.
- **ECLAT**: Reads the database only once to construct vertical TID lists, then performs fast bitwise AND intersections in RAM ($O(1)$ disk I/O). However, if transactions are long and item frequencies are high, storing millions of transaction IDs in memory can cause memory exhaustion!

Solved Problem 15: Cross-Support Property & Hyperclique Pattern Mining

When items with vastly different support levels appear together (e.g. 'Caviar' (support 0.1%) and 'Milk' (support 80%)), standard confidence generates spurious cross-support rules:

- **H-Confidence Metric**:

$$\text{h-conf}(\mathbf{X}) = \frac{\text{Support}(\mathbf{X})}{\max_{i \in \mathbf{X}} \text{Support}(\{i\})}$$

- A pattern X is a **Hyperclique Pattern** if $\text{h-conf}(X) \geq h_{\min}$. Guarantees all items in the pattern have strong mutual affinity!

Solved Problem 16: Multi-Dimensional Association Rules with Hybrid Quantitative Grid Discretization

Combines categorical attributes with numeric grid intervals to discover cross-domain rules: $\text{Age}(X, 20..29) \wedge \text{Occupation}(X, \text{'Engineer'}) \implies \text{Income}(X, \text{'High'})$ [Conf = 85%].

Solved Problem 17: Constraint-Based Mining with Convertible Anti-Monotone Constraints

Given constraint $\text{avg}(S.\text{price}) \geq 50$. When items are sorted in descending order of price, if prefix subset S' has $\text{avg}(S') < 50$, appending any further lower-priced items will only further decrease the average! The constraint becomes strictly anti-monotone and enables deep subtree pruning in FP-Growth!

Solved Problem 18: Hash-Tree Support Counting in Apriori Pass 3

To avoid checking every candidate itemset against all transactions, Apriori stores candidate k -itemsets in a **Hash Tree** where each internal node contains a hash function $h(item) = item \pmod{B}$. When matching a transaction T , the hash tree recursively routes subsets through matching hash buckets, checking candidates in $O(1)$ tree traversal time rather than linear scans!